CLIMATE INTELLIGENCE

Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning

SYSTEM DESIGN AND SOFTWARE ARCHITECTURE

Component boundaries, data contracts and persistence

Student name: _____ Roll number: _____

Class / division: _____ Course: _____ Institution: _____

1 PROBLEM, OBJECTIVES AND SCOPE

Coursework weather files use different column names and contain different variables, making records difficult to compare consistently. Climate Intelligence provides a local desktop workflow to normalize selected readings, validate them, persist them, summarize heat risk and demonstrate early-warning logic.

The objective is to connect the Python course concepts to a climate use case through a Tkinter GUI, modular processing and SQLite storage. The system supports CSV ingestion, record CRUD/search, temperature and rainfall analysis, threshold alerts, a simple trend estimate and synthetic simulation. It is an educational prototype, not a live public-warning service.

IMPLEMENTATION SNAPSHOT	CONFIRMED DESIGN
Interface	Python 3, Tkinter / ttk, Canvas-drawn visualizations
Persistence	sqlite3; one local observations table
Input / output	CSV DictReader and CSV export; XLSX companions are not imported by the GUI
Concurrency	threading, queue and ThreadPoolExecutor for local demonstration work

Scope boundary: no cloud deployment, physical sensor, weather API, geographic map service or trained machine-learning model is present.

2 ARCHITECTURE AND MODULE RESPONSIBILITIES

ClimateApp coordinates five screens and invokes the domain and persistence layers. The parser normalizes source headers; validation rejects malformed measurements; alert severity is computed before database writes. Background workers return results through a queue so Tk widgets are updated on the GUI event loop.

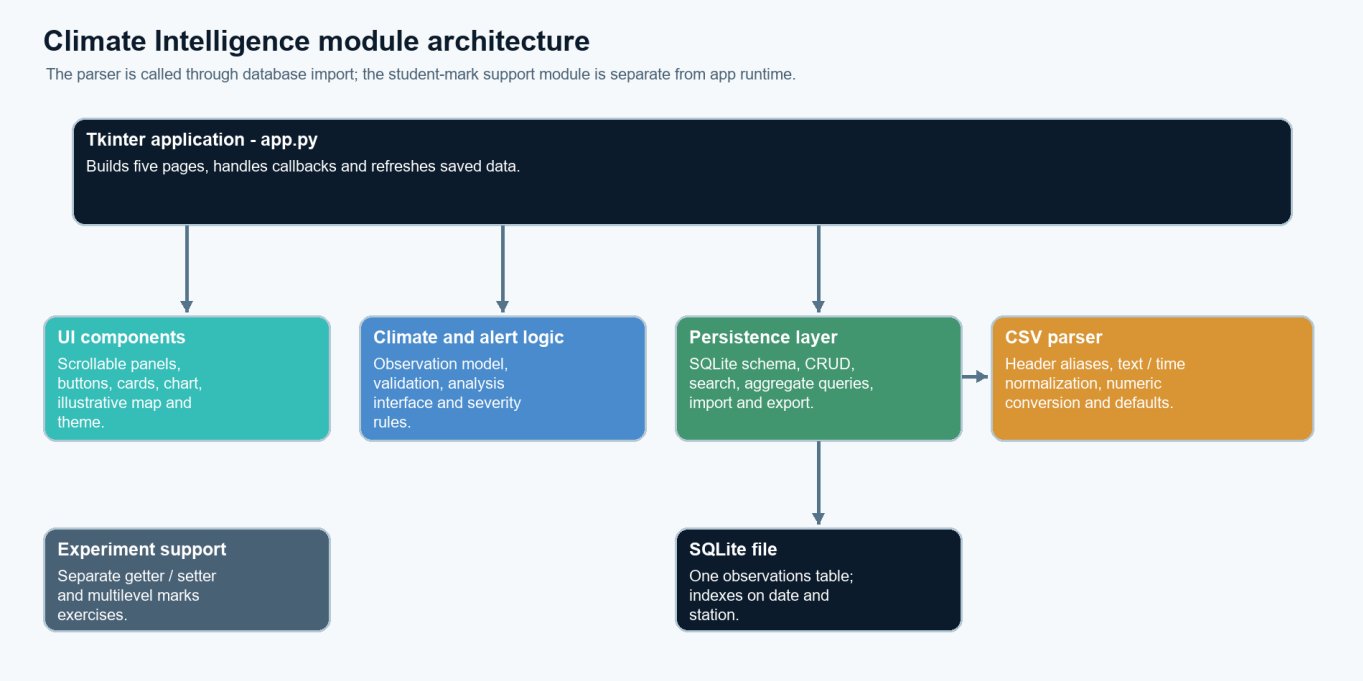


Figure 1. Source-derived module boundaries and storage path.

MODULE	ROLE IN THE RUNNING APPLICATION
app.py	Tkinter pages, callbacks, refresh, simulation and worker coordination
climate_engine.py	Observation model, validation, analyzers, Celsius conversion
weather_data.py	Header aliases, unit mapping and row normalization
alert_engine.py / database.py	Temperature severity and SQLite operations
ui_components.py / theme.py	Reusable visual components and style values

App composition · app.py

```
self._build_layout()
self._build_dashboard()
self._build_records()
self._build_analysis()
self._build_predictions()
self._build_about()
self.show_page("dashboard")
```

The application is a small layered design; app.py remains the coordinator rather than a formal MVC controller.

3 DATA INGESTION, VALIDATION AND PERSISTENCE

SUPPLIED CSV	ROWS	SELECTED USE
weather_data exp 03.csv	50	Station_ID, City, Temperature, Humidity, Condition, Wind_Speed
Exp04_IMD_50_Real_Weather_Observations.csv	50	Date, Station, Temperature_C, Rainfall_mm; source text is unverified
Pune_Climate_50_Observations EXP 6.csv	50	UTC-labelled time, station, temperature and humidity
Weather data (EXP5).csv	24,070	20 weather fields; manual import, provider not identified

Two XLSX companions are supplied but the application file picker accepts CSV only. An empty database seeds the three 50-row files (150 records). The large file is a manual import. Selected columns are normalized to one observation; source provenance is reduced to a generic label and repeated imports are not deduplicated.

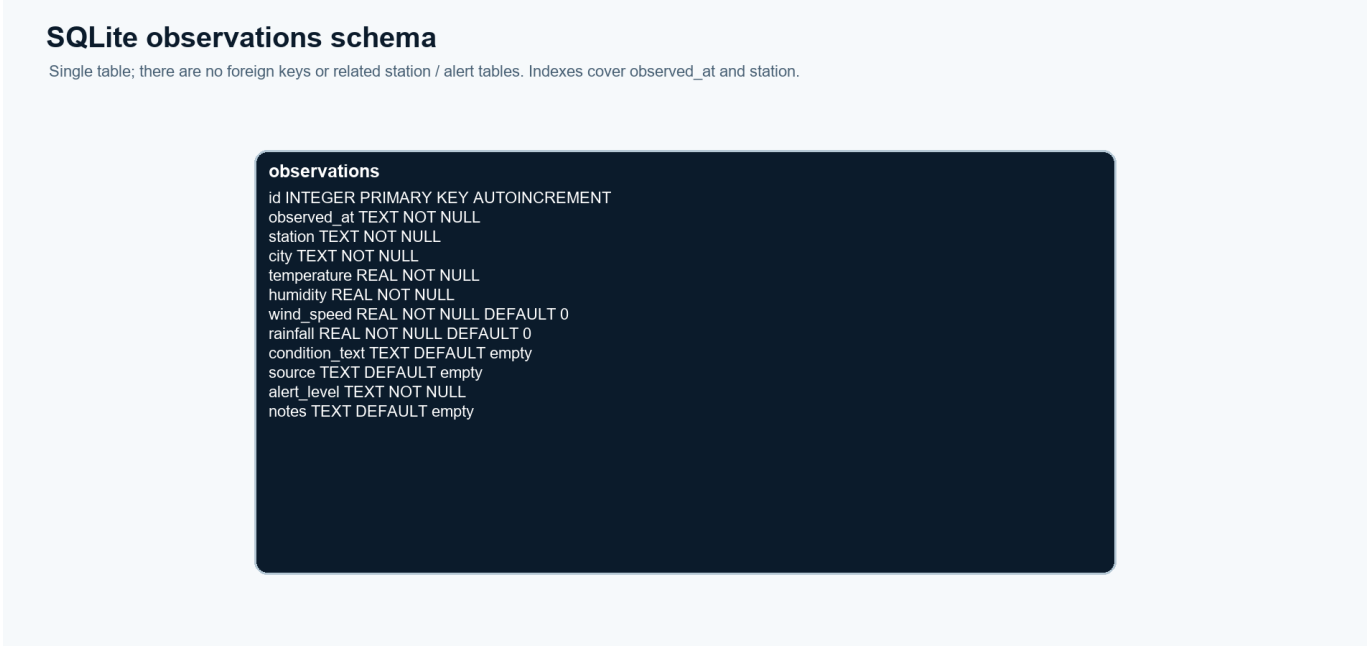


Figure 2. SQLite stores one observations table and indexes time and station.

DATABASE FACT	IMPLEMENTATION
Table	observations: id, observed_at, station, city, temperature, humidity, wind_speed, rainfall, condition_text, source, alert_level, notes
Indexes / operations	Indexes on observed_at and station; parameterized insert/update/delete, search, summary and CSV export
Validation	Required values and numeric ranges checked; temperature −20…60°C, humidity 0…100%; missing secondary CSV fields use defaults

SQLite schema · database.py

```
con.execute("""CREATE TABLE IF NOT EXISTS observations (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 observed_at TEXT NOT NULL, station TEXT NOT NULL, city TEXT NOT NULL,
 temperature REAL NOT NULL, humidity REAL NOT NULL, wind_speed REAL NOT NULL DEFAULT 0,
 rainfall REAL NOT NULL DEFAULT 0, condition_text TEXT DEFAULT '', source TEXT DEFAULT '',
 alert_level TEXT NOT NULL, notes TEXT DEFAULT '')""")
```

4 EXPERIMENT CONCEPTS INTEGRATED IN THE CODE

The table maps the six supplied coursework topics to the final implementation. Student-mark classes required by separate exercises remain support code and are not described as climate features.

EXP. / TOPIC	PYTHON CONCEPT	APPLICATION EVIDENCE
01 · Class creation	Classes, objects, constructor, attributes and encapsulation.	ClimateObservation models validated fields; StudentMarks is separate coursework support.
02 · Constructors and lists	Default/parameterized construction, list accumulation and CRUD.	weather_data.py builds row mappings; add_many validates and inserts; UI CRUD is SQLite-backed.
03 · String handling and inheritance	Whitespace/string methods, base and derived classes, super().	normalize_text and ImportedClimateObservation extend the climate model; marks inheritance is separate.
04 · Abstract classes and polymorphism	ABC, abstract method, overriding and a shared method contract.	Both analyzer subclasses implement one contract, keeping calculations replaceable.
05 · Exceptions, modules and multithreading	Exceptions, module separation, background/concurrent work.	Validation/parser errors are handled; workers and pooled checks return through a queue.
06 · Collections and functional programming	Lists, mappings, sets, lambda, map, filter and reduce.	Temperature analyzer uses mappings, collections and functional steps for heat-risk summaries.

Functional analysis · climate_engine.py

```
temperatures = list(map(lambda r: float(r["temperature"] if isinstance(r, dict) else r.temperature), observations))
hot = list(filter(lambda temp: temp >= 40.0, temperatures))
total = reduce(lambda a, b: a + b, temperatures, 0.0)
square_first = (lambda value: value * value)(temperatures[0]) if temperatures else None
return {"count": len(temperatures), "average": total / len(temperatures) if temperatures else 0.0,
 "minimum": min(temperatures) if temperatures else None,
 "maximum": max(temperatures) if temperatures else None,
 "total": total, "square_first": square_first, "hot_count": len(hot), "hot_values": hot,
 "fahrenheit": list(map(celsius_to_fahrenheit, temperatures)),
 "alert_levels": {classify_alert(temp) for temp in temperatures}}
```

The analyzer also returns average, minimum/maximum, Fahrenheit values and distinct alert categories; an empty-input branch avoids division by zero.

5 VERIFICATION, LIMITATIONS AND CONCLUSION

CHECK	OBSERVED RESULT
Observation validation	Invalid humidity 101% raises ValueError; temperature bounds are checked.
Alert boundaries	36.9°C NORMAL; 37.0°C WARNING; 40.0°C CRITICAL in unit tests.
CSV and persistence	50-row round trip passes; bootstrap stores 150 records; large CSV parser maps >20,000 rows.
GUI workflow	Smoke test checks all five pages/navigation, 880×580 geometry, simulation persistence and the demo row.
Suite result	16/16 automated tests passed on 13 Oct 2026; tests use temporary databases.

Limitations and conclusion

The design demonstrates a complete local data path, but does not retain dataset-level provenance, prevent duplicate imports or provide database migrations. The station map is illustrative. The 37°C / 40°C bands are classroom thresholds. “Prediction” is an arithmetic per-reading trend over up to eight recent values, not a trained or calibrated forecast. Synthetic simulation is not observed weather. Within that boundary, the modular GUI, validation and persistent store provide a repeatable climate-intelligence coursework demonstration.

Selected references

[1] Python Software Foundation. Python 3 Library Reference: tkinter, csv, sqlite3 and concurrent.futures. [2] SQLite Consortium, SQL Language Reference. [3] Supplied Python experiments 01–06 and project source files, preserved in the project workspace.